ALEXANDRE Ewan, Groupe 1

06/11/2026

TP N°5 : SQL ET TRIGGERS 2

Table des matières

1) Problématique 1

2) Exercices 1

Exercice 1 : Parc immobilier 1

Exercice 2 : Des produits au prix variant au fil du temps..... 4

Exercice 3 : Location de véhicules 6

3) Conclusion 8

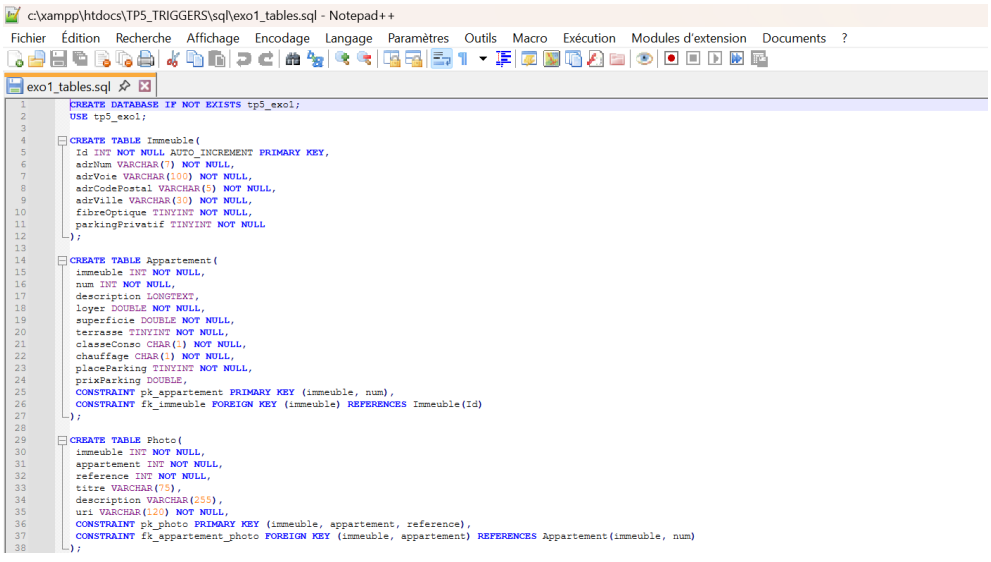
1) Problématique

Nous cherchons à concevoir et mettre en place des déclencheurs (triggers) sous MySQL afin d'automatiser des contrôles d'intégrité et de synchroniser des valeurs calculées sur plusieurs modèles relationnels. Pour cela, nous allons voir comment utiliser les événements BEFORE et AFTER pour interdire des insertions ou modifications non conformes, et recalculer automatiquement des données métiers.

2) Exercices

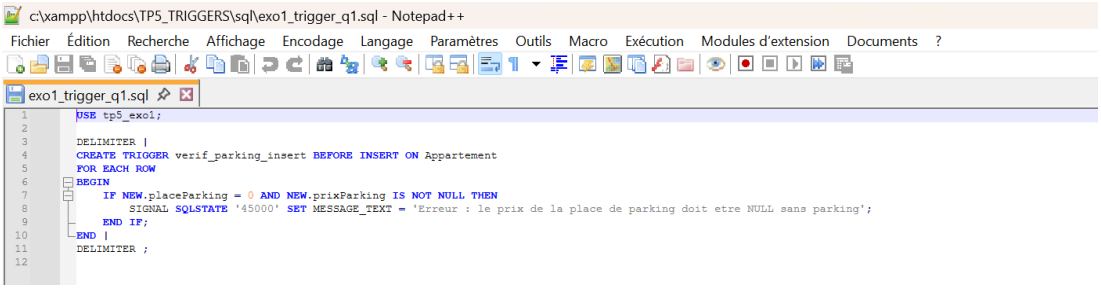
Exercice 1 : Parc immobilier

On commence par créer la base de données tp5_exo1 et les tables Immeuble, Appartement, Photo et Piece fournies dans le sujet :

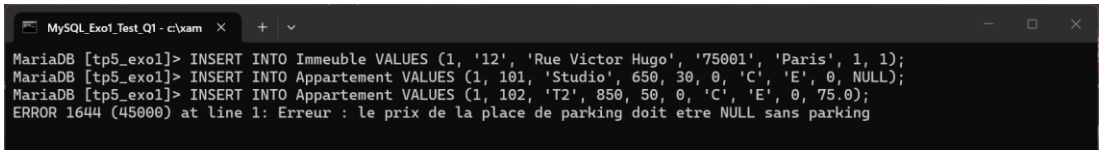


Question 1 : Trigger de contrôle du prix de parking

Ici on cherche à vérifier que si un appartement ne possède pas de place de parking (placeParking = 0), son prix de parking doit obligatoirement être NULL. S'il met un prix alors qu'il n'a pas de place, on doit bloquer l'enregistrement. Pour cela, on utilise un trigger BEFORE INSERT sur la table Appartement car on veut contrôler la valeur avant qu'elle soit insérée en base de données. Si la condition est fausse, on déclenche une erreur avec SIGNAL SQLSTATE '45000' :

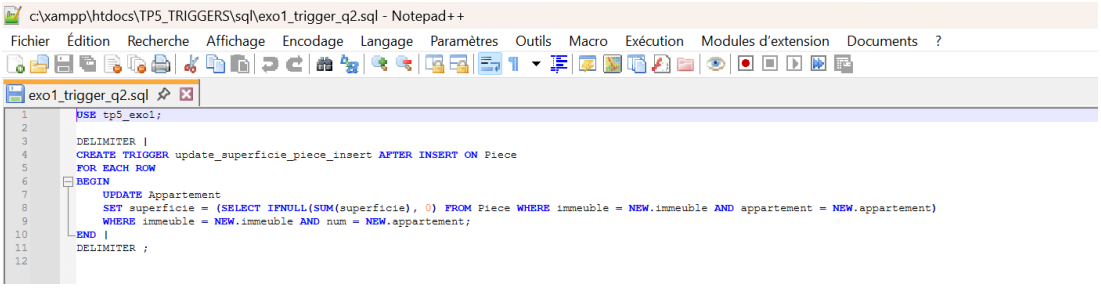


Ensuite on teste notre trigger dans le terminal MySQL en insérant d'abord un appartement valide sans place avec un prix NULL (accepté), puis un appartement invalide avec un prix de 75 alors qu'il n'a pas de place (bloqué avec le message d'erreur) :

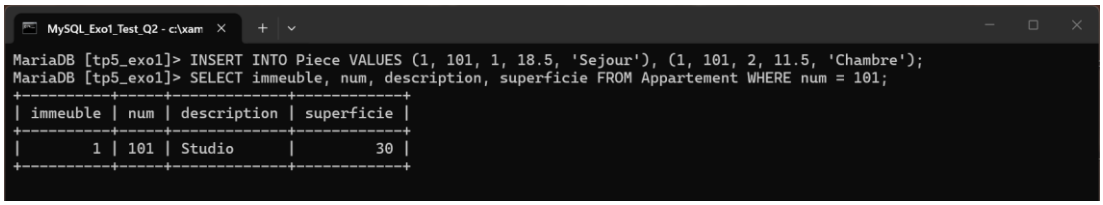


Question 2 : Calcul automatique de la superficie de l'appartement à l'insertion d'une pièce

On souhaite que la superficie totale d'un appartement soit automatiquement égale à la somme des superficies de ses pièces. Comme la superficie de l'appartement dépend des pièces qu'on y ajoute, on utilise un trigger AFTER INSERT sur la table Piece : dès qu'une nouvelle pièce est insérée, on calcule la somme des pièces de cet appartement avec SUM(superficie) et on met à jour la colonne superficie de l'appartement correspondant avec un UPDATE :

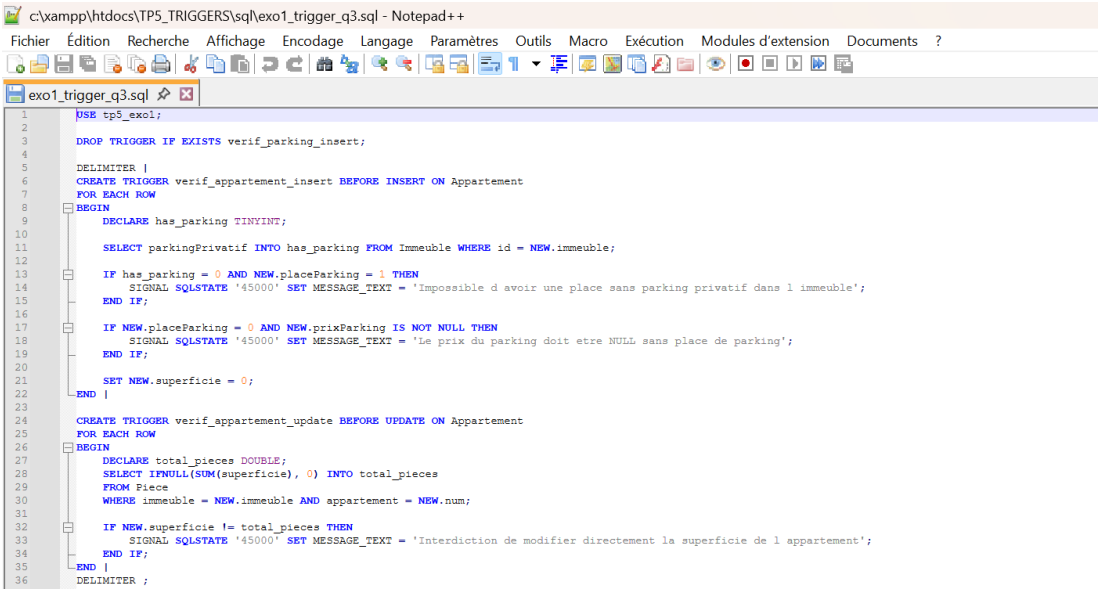


Pour tester, on insère un appartement avec une superficie de 0, puis on y ajoute deux pièces de 35 m^2 et 15 m^2. On fait ensuite un SELECT pour vérifier que la superficie totale de l'appartement s'est bien mise à jour à 50 m^2 automatiquement :

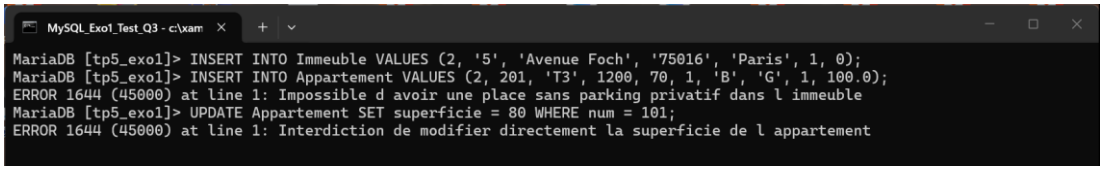


Question 3 : Adaptation des contraintes sur l'appartement

On adapte le trigger d'insertion pour ajouter trois règles supplémentaires demandées :
1. Si l'appartement a une place de parking (placeParking = 1), on vérifie dans la table Immeuble que l'immeuble possède bien un parking privatif (parkingPrivatif = 1). Sinon, on bloque.
2. À l'insertion d'un appartement, on force sa superficie à 0 avec SET NEW.superficie = 0; car elle sera calculée à partir des pièces.
3. En cas de modification d'un appartement, on empêche l'utilisateur de changer manuellement la superficie en créant un trigger BEFORE UPDATE qui bloque la modification si NEW.superficie <> OLD.superficie :



On teste ces règles dans le terminal en essayant d'attribuer une place de parking dans un immeuble sans parking privatif (rejeté), puis en tentant de modifier directement la superficie avec un UPDATE (rejeté par le trigger) :



Question 4 : Mise à jour et suppression de pièces

Si on modifie la superficie d'une pièce ou si on supprime une pièce, la superficie totale de l'appartement doit s'ajuster en conséquence. On crée donc deux triggers supplémentaires sur la table Piece :
- Un trigger AFTER UPDATE qui recalcule la somme après modification d'une pièce.
- Un trigger AFTER DELETE qui recalcule la somme après suppression d'une pièce en utilisant OLD.immeuble et OLD.appartement avec IFNULL(SUM(superficie), 0) si l'appartement n'a plus de pièce :

c:\xampp\htdocs\TP5_TRIGGERS\sql\exo1_trigger_q4.sql - Notepad++

Fichier Édition Recherche Affichage Encodage Langage Paramètres Outils Macro Exécution Modules d'extension Documents ?

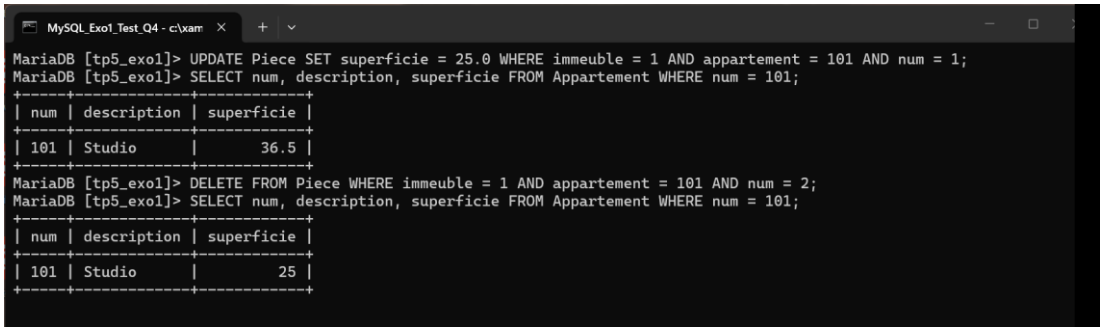
exo1_trigger_q4.sql

```sql
USE tp5_exo1;

DELIMITER |
CREATE TRIGGER update_superficie_piece_update AFTER UPDATE ON Piece
FOR EACH ROW
BEGIN
    UPDATE Appartement
    SET superficie = (SELECT IFNULL(SUM(superficie), 0) FROM Piece WHERE immeuble = NEW.immeuble AND appartement = NEW.appartement)
    WHERE immeuble = NEW.immeuble AND num = NEW.appartement;
END |

CREATE TRIGGER update_superficie_piece_delete AFTER DELETE ON Piece
FOR EACH ROW
BEGIN
    UPDATE Appartement
    SET superficie = (SELECT IFNULL(SUM(superficie), 0) FROM Piece WHERE immeuble = OLD.immeuble AND appartement = OLD.appartement)
    WHERE immeuble = OLD.immeuble AND num = OLD.appartement;
END |
DELIMITER ;
```

On teste dans le terminal en modifiant la taille d'une pièce (de 20 m^2 à 25 m^2), puis en supprimant une pièce, et on vérifie à chaque étape la superficie de l'appartement avec un SELECT :



```
MySQL_Exo1_Test_Q4 - c:\xam
MariaDB [tp5_exo1]> UPDATE Piece SET superficie = 25.0 WHERE immeuble = 1 AND appartement = 101 AND num = 1;
MariaDB [tp5_exo1]> SELECT num, description, superficie FROM Appartement WHERE num = 101;
+-----+-------------+------------+
| num | description | superficie |
+-----+-------------+------------+
| 101 | Studio      |       36.5 |
+-----+-------------+------------+
MariaDB [tp5_exo1]> DELETE FROM Piece WHERE immeuble = 1 AND appartement = 101 AND num = 2;
MariaDB [tp5_exo1]> SELECT num, description, superficie FROM Appartement WHERE num = 101;
+-----+-------------+------------+
| num | description | superficie |
+-----+-------------+------------+
| 101 | Studio      |         25 |
+-----+-------------+------------+
```

Exercice 2 : Des produits au prix variant au fil du temps

Question 1 : Script de création des tables

On rédige le script de création de la base de données tp5_exo2 avec les tables Produit, Mode_Livraison, Frais_Transport, Commande et Ligne_Commande en déclarant les clés primaires et les clés étrangères nécessaires :

c:\xampp\htdocs\TP5_TRIGGERS\sql\exo2_tables.sql - Notepad++

Fichier Édition Recherche Affichage Encodage Langage Paramètres Outils Macro Exécution Modules d'extension Documents ?

exo2_tables.sql

```sql
CREATE DATABASE IF NOT EXISTS tp5_exo2;
USE tp5_exo2;

CREATE TABLE Produit(
  code INT NOT NULL PRIMARY KEY,
  libelle VARCHAR(100) NOT NULL,
  prix DOUBLE NOT NULL,
  formation TINYINT NOT NULL,
  poids DOUBLE NOT NULL
);

CREATE TABLE Mode_Livraison(
  num INT NOT NULL PRIMARY KEY,
  libelle VARCHAR(100) NOT NULL
);

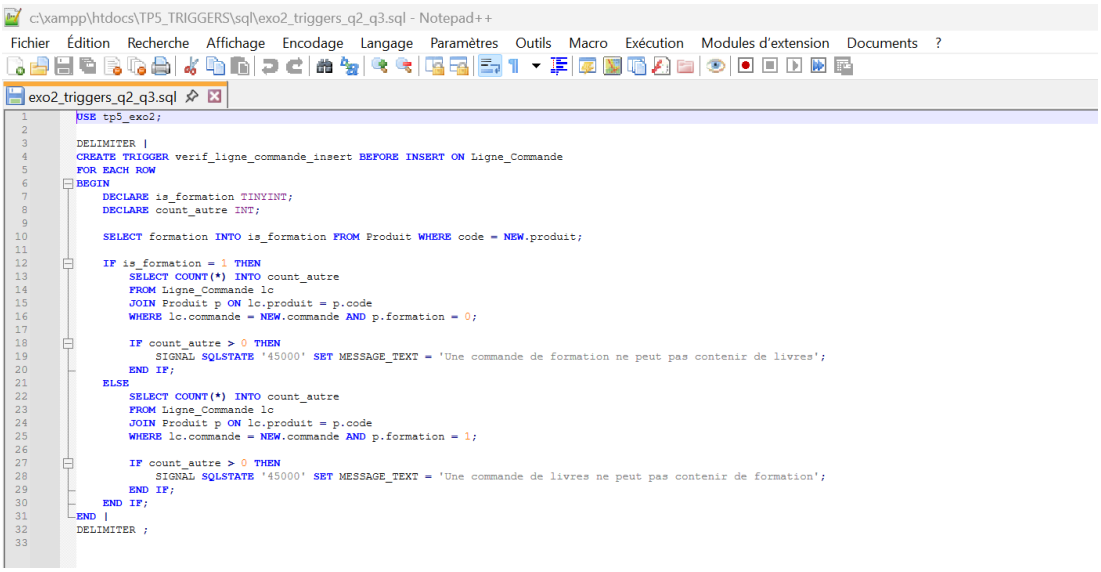
CREATE TABLE Frais_Transport(
  livraison INT NOT NULL,
  num INT NOT NULL,
  prix DOUBLE NOT NULL,
  poidsMin DOUBLE NOT NULL,
  poidsMax DOUBLE NOT NULL,
  CONSTRAINT pk_frais_transport PRIMARY KEY (livraison, num),
  CONSTRAINT fk_frais_mode FOREIGN KEY (livraison) REFERENCES Mode_Livraison(num)
);

CREATE TABLE Commande(
  num INT NOT NULL PRIMARY KEY,
  destinataire VARCHAR(100) NOT NULL,
  prixTotal DOUBLE,
  livraison INT,
  transport INT,
  CONSTRAINT fk_commande_transport FOREIGN KEY (livraison, transport) REFERENCES Frais_Transport(livraison, num)
);

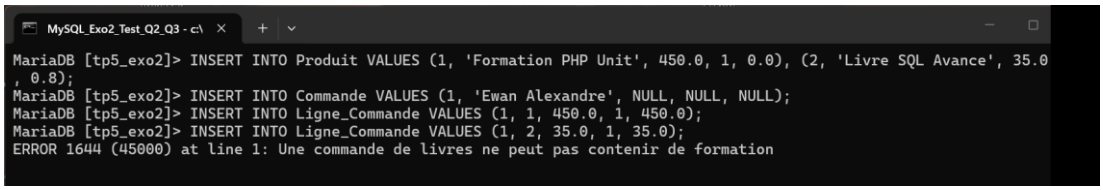
CREATE TABLE Ligne_Commande(
  commande INT NOT NULL,
  produit INT NOT NULL,
```

Questions 2 et 3 : Exclusion mutuelle entre formation et livres

Une commande ne peut pas mélanger des formations et des livres : si elle contient une formation, elle ne peut pas avoir de livre, et inversement. Pour contrôler cela avant l'ajout de chaque ligne, on utilise un trigger BEFORE INSERT sur la table Ligne_Commande :
On regarde si le produit ajouté est une formation (formation = 1) ou un livre (formation = 0). Ensuite, on compte s'il existe déjà dans la même commande des produits du type opposé. Si le total est supérieur à 0, on bloque l'insertion avec un message d'erreur explicite :



On teste dans le terminal en créant une commande avec une formation, puis en essayant d'y insérer un livre : le trigger bloque immédiatement la requête :



Questions 4 et 5 : Frais de transport et mise à jour automatique du prix total

On adapte le fonctionnement de la commande avec deux règles :
- Pour la question 4, une formation ne nécessite pas de livraison physique, donc elle ne doit pas comporter de frais de transport (livraison IS NULL). On rajoute cette vérification dans le trigger BEFORE INSERT.
- Pour la question 5, chaque fois qu'une ligne de commande est insérée, on recalcule automatiquement le prix total de la commande avec un trigger AFTER INSERT sur Ligne_Commande qui additionne le montant de tous les articles (SUM(prixt)) et les éventuels frais de transport dans la table Commande :

c:\xampp\htdocs\TP5_TRIGGERS\sql\exo2_triggers_q4_q5.sql - Notepad++

Fichier Édition Recherche Affichage Encodage Langage Paramètres Outils Macro Exécution Modules d'extension Documents ?

exo2_triggers_q4_q5.sql

```sql
USE tp5_exo2;

DROP TRIGGER IF EXISTS verif_ligne_commande_insert;

DELIMITER |
CREATE TRIGGER verif_ligne_commande_insert BEFORE INSERT ON Ligne_Commande
FOR EACH ROW
BEGIN
    DECLARE is_formation TINYINT;
    DECLARE count_livres INT;
    DECLARE has_livraison INT;

    SELECT formation INTO is_formation FROM Produit WHERE code = NEW.produit;

    IF is_formation = 1 THEN
        SELECT COUNT(*) INTO count_livres
        FROM Ligne_Commande lc
        JOIN Produit p ON lc.produit = p.code
        WHERE lc.commande = NEW.commande AND p.formation = 0;

        IF count_livres > 0 THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Une commande de formation ne peut pas contenir de livres';
        END IF;

        SELECT livraison INTO has_livraison FROM Commande WHERE num = NEW.commande;
        IF has_livraison IS NOT NULL THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Une commande de formation ne peut pas comporter de frais de transport';
        END IF;
    ELSE
        SELECT COUNT(*) INTO count_livres
        FROM Ligne_Commande lc
        JOIN Produit p ON lc.produit = p.code
        WHERE lc.commande = NEW.commande AND p.formation = 1;

        IF count_livres > 0 THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Une commande de livres ne peut pas contenir de formation';
        END IF;
    END IF;
```

On teste dans la console MySQL le calcul automatique du prix total lors de l'ajout d'articles avec des frais de transport associés :

MySQL_Exo2_Test_Q4_Q5 - c:\

```
MariaDB [tp5_exo2]> INSERT INTO Mode_Livraison VALUES (1, 'Colissimo');
MariaDB [tp5_exo2]> INSERT INTO Frais_Transport VALUES (1, 1, 8.5, 0.0, 2.0);
MariaDB [tp5_exo2]> INSERT INTO Commande VALUES (2, 'Client Livres', NULL, 1, 1);
MariaDB [tp5_exo2]> INSERT INTO Ligne_Commande VALUES (2, 2, 35.0, 2, 70.0);
MariaDB [tp5_exo2]> SELECT num, destinataire, prixTotal FROM Commande WHERE num = 2;
+-----+---------------+-----------+
| num | destinataire  | prixTotal |
+-----+---------------+-----------+
|   2 | Client Livres |      78.5 |
+-----+---------------+-----------+
```

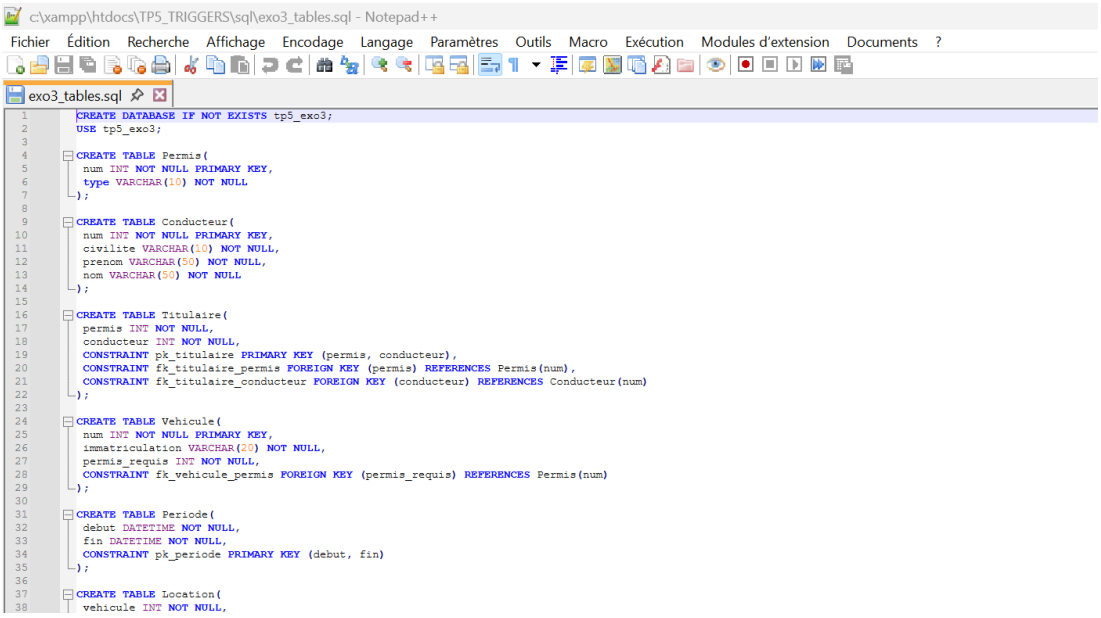
Question 6 : Cas non traités par les triggers

Au travers des questions 1 à 5, plusieurs situations de gestion réelles n'ont pas été traitées :
- Les modifications (UPDATE) sur la table Ligne_Commande : si on modifie la quantité ou le produit d'une ligne existante, le prix total de la commande ne sera pas recalculé.
- Les suppressions (DELETE) d'une ligne de commande : si un client retire un article de son panier, le prix total ne diminue pas automatiquement.
- La modification ultérieure des frais de transport dans Commande : un client pourrait créer une commande de formation sans livraison, puis modifier sa commande après coup pour y ajouter un mode de transport payant.
- L'évolution du tarif des produits dans la table Produit : si le tarif d'un produit change, les commandes passées doivent conserver leur prix historique et ne pas être impactées rétroactivement.

Exercice 3 : Location de véhicules

Question 1 : Script de création des tables

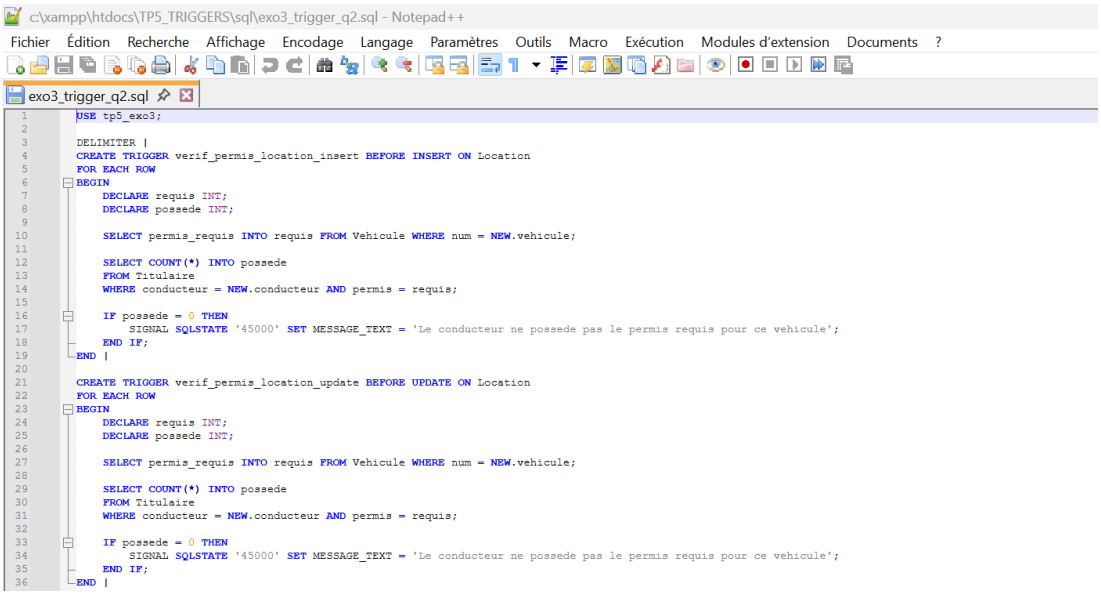
On crée les tables du modèle sous MySQL pour la gestion du parc de véhicules : Permis, Conducteur, Titulaire (table d'association n,n pour les permis d'un conducteur), Vehicule (avec clé étrangère vers le permis nécessaire), Periode, et Location :



Question 2 : Trigger de vérification du permis requis pour la location

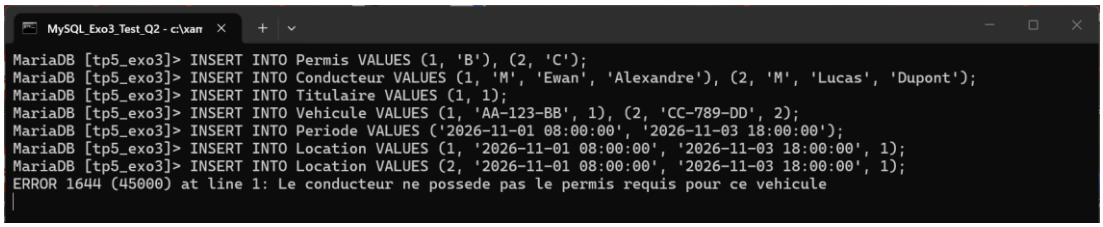
Avant d'enregistrer une location ou de la modifier (BEFORE INSERT et BEFORE UPDATE sur la table Location), on doit vérifier que le conducteur détient bien le permis exigé par le véhicule loué.

Pour cela, on cherche quel permis est nécessaire pour ce véhicule (permis_requis), puis on vérifie dans la table Titulaire si le conducteur possède ce permis précis. Si le conducteur ne possède pas ce permis (possede = 0), on bloque la location avec un message d'erreur :



On teste dans le terminal avec le conducteur Ewan qui possède le permis B :

- La location d'une voiture demandant le permis B est acceptée.
- La location d'un poids lourd demandant le permis C est bloquée par le trigger avec le message d'erreur :



3) Conclusion

En conclusion, ce TP m'a permis d'approfondir la mise en oeuvre des déclencheurs SQL sous MySQL. J'ai pu vérifier concrètement l'intérêt des événements BEFORE pour bloquer les données invalides à la source et des événements AFTER pour synchroniser automatiquement les calculs (superficie d'un appartement, prix total d'une commande) sans faire reposer cette logique sur l'application.